



Google Firebase

தமிழில் முழுமையான அடிப்படை வழிகாட்டி

Frontend Developers-க்காக எளிமையான முறையில்

Backend இல்லாமல் App கட்டலாம் — Firebase இருக்கு!

இந்த **guide**-ல் என்ன இருக்கு?

1. Firebase என்றால் என்ன?
2. Firebase Setup (3 படிகள்)
3. Firestore Database — முழு CRUD
4. collection() vs doc() — எப்போது எது?
5. Real-time Listener
6. Authentication — Login System
7. Security Rules
8. அடுத்து என்ன படிக்கணும்?

Firestore என்ன?

Firestore என்பது **Google** தரும் ஒரு **Backend-as-a-Service (BaaS)**. Frontend developers-க்கு தனியாக server எழுத தேவையில்லை — Firestore அதை செய்துவிடும்.

எளிமையாக புரிஞ்சுக்க: நீங்கள் கட்டிடம் கட்டணும் என்றால் — பூமி, செங்கல், cement எல்லாம் தனியா வாங்கணும். Firestore என்னன்னா, கட்டத்தயாரான plot கொடுக்கும். நீங்கள் வீடு மட்டும் design பண்ணுங்க!

Firestore-ல் என்னென்ன கிடைக்கும்?

 Firestore Database முக்கிய Database (NoSQL) Real-time updates கிடைக்கும் புதிய app-க்கு இதை பயன்படுத்து
 Authentication Login system (Google, Email...) 5 நிமிடத்தில் setup ஆகும் Password manage செய்ய வேண்டாம்
 Storage Image / File upload Profile photo, documents S3 மாதிரி, ஆனால் எளிமையானது
 Hosting Frontend app deploy செய்ய Free SSL certificate CDN globally available

யாருக்கு Firestore useful?

- Backend தெரியாத Frontend developers
- Quick prototype கட்ட வேண்டியவர்கள்

- Solo developers, small teams
- Chat app, todo app, portfolio போன்றவை கட்ட

Firebase Setup — 3 படிகள்

1 Firebase SDK Install செய்யுங்கள்

உங்கள் project folder-ல் terminal திறந்து:

```
npm install firebase
```

2 Firebase Console-ல் Project உருவாக்குங்கள்

console.firebase.google.com → New Project → உங்கள் config object எடுங்கள்

```
// Firebase Console-ல் இருந்து கிடைக்கும்
const firebaseConfig = {
  apiKey: "AIzaSy...",
  authDomain: "your-app.firebaseio.com",
  projectId: "your-app",
  storageBucket: "your-app.appspot.com",
  messagingSenderId: "123456789",
  appId: "1:123456:web:abc123"
};
```

3 App-ல் Initialize செய்யுங்கள்

உங்கள் main file-ல் (App.js அல்லது firebase.js):

```
import { initializeApp } from "firebase/app";
import { getFirestore } from "firebase/firestore";

const app = initializeApp(firebaseConfig);
const db = getFirestore(app);

export { db }; // மற்ற files-ல் import செய்ய
```

Tip: firebaseConfig-ஐ தனி ஒரு file-ல் வை (firebase.js). எல்லா components-லும் இந்த file-ல் இருந்து db import பண்ணலாம்.

Firestore Database — அமைப்பு

Firestore என்பது SQL மாதிரி table இல்லை. இது **document** அடிப்படையில் வேலை செய்யும்.

3 Level Structure

Collection → ஒரு folder மாதிரி (பல documents-ஐ hold பண்ணும்)
Document → அந்த folder-ல் உள்ள ஒரு file மாதிரி
Field → அந்த file-ல் உள்ள actual data (key: value)

Real உதாரணம்

```
users/                                ← collection (folder)
  uid_abc123/                         ← document (file)
    name: "ரவி"                      ← field
    email: "ravi@gmail.com"          ← field
    age: 25                          ← field

  uid_def456/                        ← இன்னொரு document
    name: "கவிதா"
    email: "kavitha@gmail.com"
    age: 30

posts/                                ← இன்னொரு collection
  post_001/
    title: "வணக்கம் உலகம்"
    author: "uid_abc123"
    likes: 10
```

SQL vs Firestore:

SQL: Table → Row → Column

Firestore: Collection → Document → Field

SQL-ல் JOIN பண்ணணும். Firestore-ல் UI-க்கு தேவையான மாதிரி data வைக்கலாம்.

Firestore CRUD — முழு விளக்கம்

CREATE — Data சேர்க்க

Function	என்ன செய்யும்	எப்போது பயன்படுத்து
addDoc()	Auto ID போட்டு சேர்க்கும்	ID தெரியாத போது
setDoc()	நீயே ID குடுத்து சேர்க்கலாம்	Custom ID வேணும் போது

```
import { collection, addDoc, doc, setDoc } from "firebase/firestore";

// 1. Auto ID — Firebase தானே போடும்
await addDoc(collection(db, "users"), {
  name: "ரவி",
  email: "ravi@gmail.com",
  age: 25
});

// 2. Custom ID — நீயே போடுவ
await setDoc(doc(db, "users", "myCustomId_001"), {
  name: "கவிதா",
  email: "kavitha@gmail.com"
});
```

READ — Data படிக்க

Function	என்ன செய்யும்	எப்போது பயன்படுத்து
getDoc()	ஒரு document மட்டும் எடுக்கும்	Specific user profile
getDocs()	Collection-ல் எல்லாமே	Users list காட்ட
onSnapshot()	Real-time, data மாறும்போது auto update	Chat, live feed

```
import { doc, getDoc, collection, getDocs, onSnapshot } from "firebase/firestore"
;

// 1. ஒரு document மட்டும் எடு
const snap = await getDoc(doc(db, "users", "uid_abc123"));
```

```

if (snap.exists()) {
  console.log(snap.data()); // { name: "ரவி", email: "..."}
}

// 2. எல்லா documents-ஊம் எடு
const snap = await getDocs(collection(db, "users"));
snap.forEach(doc => {
  console.log(doc.id, doc.data());
});

// 3. Real-time – data மாறும்போது தானே வரும்
onSnapshot(collection(db, "messages"), (snap) => {
  snap.forEach(doc => console.log(doc.data()));
});

```

UPDATE — Data மாற்ற

Function	என்ன செய்யும்	Risk
updateDoc()	சில fields மட்டும் மாற்றும்	Safe ✓
setDoc() with merge:true	Partial update (alternative)	Safe ✓
setDoc() without merge	முழு document replace ஆகும்	பழைய data போகும் ⚠

```

import { doc, updateDoc, setDoc } from "firebase/firestore";

// 1. Safe update – age மட்டும் மாறும், name தொடரது
await updateDoc(doc(db, "users", "uid_abc123"), {
  age: 26
});

// 2. setDoc with merge – updateDoc மாதிரியே safe
await setDoc(doc(db, "users", "uid_abc123"), {
  age: 26
}, { merge: true });

// 3. setDoc without merge – FULL replace! பழைய data போயிடும்
await setDoc(doc(db, "users", "uid_abc123"), {
  name: "ரவி",
  age: 26
// name மட்டும் வச்ச பழைய email, phone எல்லாம் போயிடும்!
});

```

DELETE — Data அழிக்க

```
import { doc, deleteDoc, updateDoc, deleteField } from "firebase/firestore";

// 1. முழு document அழி
await deleteDoc(doc(db, "users", "uid_abc123"));

// 2. ஒரு field மட்டும் அழி (document இருக்கும்)
await updateDoc(doc(db, "users", "uid_abc123"), {
  age: deleteField() // age போகும், name/email இருக்கும்
});
```

SPECIAL Operations — Increment & Array

```
import { increment, arrayUnion, arrayRemove } from "firebase/firestore";

await updateDoc(doc(db, "posts", "post_001"), {
  likes: increment(1),           // likes = likes + 1
  likes: increment(-1),         // likes = likes - 1
  tags: arrayUnion("firebase"), // tags array-ல் "firebase" சேர்
  tags: arrayRemove("old-tag")  // tags array-ல் இருந்து "old-tag" நீக்கு
});
```

increment() ஏன் special?

நீங்கள் முதல் likes value-ஐ fetch பண்ணி, +1 கூட்டி, update பண்ணினா — 2 users ஒரே நேரத்தில் like பண்ணினா race condition வரும். increment() Firebase-லயே atomically கூட்டும், safe!

collection() vs doc() — ஏன் இப்படி?

இந்த pattern-ஐ பார்க்கும்போது initially confuse ஆகும். ஆனால் ஒரே ஒரு rule புரிஞ்சுட்டா போதும்:

collection() = folder level | doc() = file level

Folder vs File — Analogy

collection(db, "users")

→ "users" என்னும் முழு folder-ஐ சுட்டுது
(எல்லா users-உம் வேண்டும் / புதுசா எங்க சேர்க்கணும்னு)

doc(db, "users", "uid_abc123")

→ "users" folder-ல் "uid_abc123" என்னும் specific file-ஐ சுட்டுது
(ஒரு குறிப்பிட்ட user மட்டும் வேண்டும்)

எந்த operation எதை பயன்படுத்தும்?

Operation	பயன்படுத்துவது	ஏன்?
addDoc()	collection()	எந்த collection-ல் சேர்க்கணும்னு சொல்லணும்
getDocs()	collection()	எல்லா documents-உம் வேண்டும்
getDoc()	doc()	ஒரு specific document மட்டும்
updateDoc()	doc()	எதை update பண்ணோம்னு தெரியணும்
deleteDoc()	doc()	எதை delete பண்ணோம்னு தெரியணும்
setDoc()	doc()	எந்த document-ஐ set பண்ணோம்னு தெரியணும்

ஒரே ஒரு document தொட்டா → doc() | பல / எல்லாமே → collection()

Real-time Listener — Firebase-ன் சிறப்பு திறன்

இது Firebase-ன் மிகவும் powerful feature. Database-ல் data மாறும்போது **page reload** இல்லாமல் உங்கள் app தானாக update ஆகும்.

getDoc() vs onSnapshot() — வித்தியாசம்

	getDoc() / getDocs()	onSnapshot()
எப்போது run ஆகும்?	ஒரு முறை மட்டும்	data மாறும் ஒவ்வொரு முறையும்
Use case	Profile page load	Chat, live notifications
Cost	ஒரு read	மாறும்போது ஒவ்வொரு முறையும் read

```
import { collection, onSnapshot } from "firebase/firestore";

// Real-time listener setup
const unsubscribe = onSnapshot(collection(db, "messages"), (snap) => {
  const messages = snap.docs.map(doc => ({
    id: doc.id,
    ...doc.data()
  }));
  console.log(messages); // database மாறும்போது இது run ஆகும்
});

// Listener நிறுத்த (component unmount ஆகும்போது)
unsubscribe();
```

React-ல் பயன்படுத்த: useEffect-ல் onSnapshot வை. cleanup function-ல் unsubscribe() call பண்ணு. இல்லன்னா memory leak வரும்!

Authentication — Login System

Firebase Authentication-ஐ பயன்படுத்தி 5 நிமிடத்தில் Login system உருவாக்கலாம். Password hash, token management எல்லாம் Firebase பார்த்துக்கும்.

Google Login Setup

```
import { getAuth, signInWithPopup, GoogleAuthProvider, signOut } from "firebase/auth";

const auth = getAuth(app);
const provider = new GoogleAuthProvider();

// 1. Google Popup மூலம் Login
const result = await signInWithPopup(auth, provider);
console.log(result.user.uid);           // unique user ID
console.log(result.user.email);         // email
console.log(result.user.displayName);   // பெயர்

// 2. தற்போதைய user யார்?
const user = auth.currentUser;
if (user) {
  console.log("Logged in:", user.email);
} else {
  console.log("Not logged in");
}

// 3. Logout
await signOut(auth);
```

Auth State Change — User Login/Logout detect பண்ண

```
import { onAuthStateChanged } from "firebase/auth";

onAuthStateChanged(auth, (user) => {
  if (user) {
    console.log("User logged in:", user.uid);
  } else {
    console.log("User logged out");
  }
});
```

```
}  
});
```

Auth + Firestore இணைப்பு

```
// Login ஆனவுடன் Firestore-ல் user data save பண்ணு  
const result = await signInWithPopup(auth, provider);  
const user = result.user;  
  
await setDoc(doc(db, "users", user.uid), {  
  name: user.displayName,  
  email: user.email,  
  photo: user.photoURL,  
  lastLogin: new Date()  
}, { merge: true }); // merge: true – existing data கெட்டுவிடாது
```

ஏன் **user.uid**-ஐ **document ID**-ஆக வைக்கணும்?

user.uid unique. ஒவ்வொரு user-க்கும் அவங்க uid-ஐ வச்ச document create பண்ணினா, "என்னுடைய data மட்டும் கொடு" என்று எளிதாக query பண்ணலாம். Security rules-லும் இதை check பண்ணலாம்.

Security Rules — மிக முக்கியம்!

Firebase Console → Firestore → Rules-ல் எழுதுவது. இது யாருக்கு எந்த data-வை access கொடுக்கணும்னு control பண்ணும்.

⚠ **Warning:** Default-ஆக test mode-ல் allow read, write: if true இருக்கும். இது எல்லாருக்கும் open! Production-க்கு போவதற்கு முன் Rules மாத்தணும்.

Basic Security Rules

```
// Firebase Console-ல் Rules tab-ல் paste பண்ணுங்கள்

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // யாரும் படிக்க முடியாது — default deny

    // Login ஆனவங்கள் மட்டும் தங்களுடைய data
    match /users/{userId} {
      allow read, write: if request.auth != null
                          && request.auth.uid == userId;
    }

    // எல்லாரும் posts படிக்கலாம், write-க்கு login வேணும்
    match /posts/{postId} {
      allow read: if true;
      allow write: if request.auth != null;
    }
  }
}
```

Rule	என்ன அனுமதிக்கும்
if true	யாரும் (dangerous!)
if false	யாரும் இல்லை
if request.auth != null	Login ஆனவங்கள் மட்டும்

if request.auth.uid == userId

அந்த document owner மட்டும்

CRUD — Full Summary & அடுத்த படிகள்

CRUD Functions — ஒரே பார்வையில்

CREATE

`addDoc(collection())` → Auto ID, collection-ல் சேர்
`setDoc(doc())` → Custom ID, நீயே போடு

READ

`getDoc(doc())` → ஒரு document (one-time)
`getDocs(collection())` → எல்லா மே (one-time)
`onSnapshot()` → Real-time, auto update

UPDATE

`updateDoc(doc())` → Partial update (SAFE ✓)
`setDoc(merge:true)` → Partial update (alternative)
`setDoc()` → Full replace (பழையது போகும் Δ)

DELETE

`deleteDoc(doc())` → முழு document அழி
`deleteField()` → ஒரு field மட்டும் அழி

SPECIAL

`increment(n)` → Number atomically கூட்டு/கழி
`arrayUnion(val)` → Array-ல் சேர்
`arrayRemove(val)` → Array-ல் இருந்து நீக்கு

அடுத்து என்ன கத்துக்கணும்?

1 Firestore Querying

`where()`, `orderBy()`, `limit()` — filter செய்து data எடுக்க

2 Auth + Firestore இணைப்பு (Practice)

Login ஆன user-ன் data மட்டும் show / save பண்ண

3 Security Rules — Deep Dive

Production-க்கு ready-ஆக rules எழுத கத்துக்க

4 Storage — File / Image Upload

Profile photo upload, documents store பண்ண

5 First Real App கட்டுங்கள்!

Todo App அல்லது simple Chat App — CRUD + Auth சேர்த்து

Google Firebase — தமிழில் அடிப்படை வழிகாட்டி

Firestore

Auth

Real-time

Security Rules

Firebase documentation: firebase.google.com/docs